Validaciones en Java y Spring Boot

Qué tipos hay, dónde se pone cada una y por qué · con ejemplos de la biblioteca

> **La idea en una frase**
> Cada validación va en la capa que **tiene la información necesaria** para comprobarla. Si para saber si el dato es válido basta con mirar el propio objeto, va en el objeto (constructor) o en el DTO. Si hace falta mirar *otros* datos (¿existe ya ese ISBN?, ¿existe esa categoría?), va en el servicio.

1. Mapa general: qué va en cada capa
2. En el constructor y el modelo
3. En el DTO con Bean Validation
4. En el controlador (ruta y parámetros)
5. En el servicio (reglas de negocio)
6. Validador propio
7. Grupos: crear y modificar
8. En la configuración
9. En la base de datos
10. Qué pasa cuando falla: errores HTTP
11. Chuleta final y errores típicos

1 Mapa general: qué va en cada capa

Capa	Qué se valida ahí	Cómo	Si falla
Modelo (constructor, setters)	Lo que tiene que cumplirse **siempre** para que el objeto tenga sentido (invariantes)	`if` + excepción, `Objects.requireNonNull`, constructor compacto de un `record`	`IllegalArgumentException` o excepción propia
DTO de entrada	Forma del dato que manda el cliente: obligatorio, longitud, rango, formato	Anotaciones de Bean Validation (`@NotBlank`, `@Size`…) + `@Valid`	400 Bad Request
Controlador	Parámetros de ruta y de consulta: tipo, obligatorio, rango	`@PathVariable`, `@RequestParam` y anotaciones sobre ellos	400 Bad Request
Servicio	Reglas de negocio que dependen de otros datos: no repetido, existe, estado permitido	`if` + excepción propia	404, 409 o 400 según la regla
Configuración	Que las propiedades de `application.properties` sean correctas	`@ConfigurationProperties` + `@Validated`	La aplicación no arranca
Base de datos	Última barrera: no nulo, único, longitud, clave ajena	`@Column(nullable = false, unique = true)`	Excepción de integridad → 409

> **El controlador no decide reglas**
> El controlador solo recibe la petición, activa la validación del DTO con `@Valid` y llama al servicio. Un `if` con una regla de negocio dentro de un controlador es justo lo que se penaliza como «regla en la capa equivocada».

2 En el constructor y el modelo

Cuándo: cuando el dato es obligatorio para que el objeto exista. Si una `Persona` sin edad válida no tiene sentido, no se debe poder construir. Así es imposible tener en memoria un objeto «a medias», lo cree quien lo cree (un servicio, un test, un fichero JSON).

Validación de edad en el constructor

```
public class Persona {
    private final String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre es obligatorio");
        }
        this.nombre = nombre;
        setEdad(edad);          // se reutiliza la validación del setter
    }

    public void setEdad(int edad) {
        if (edad < 0 || edad > 130) {
            throw new IllegalArgumentException("La edad debe estar entre 0 y 130");
        }
        this.edad = edad;
    }
}
```

- Si el atributo se puede cambiar después, la comprobación va **en el setter** y el constructor llama al setter. Si solo estuviera en el constructor, un `setEdad(-5)` posterior se la saltaría.
- Si el atributo es `final` (no cambia nunca), basta con validarlo en el constructor.

Formas cortas

```
// 1) Objects.requireNonNull: lanza NullPointerException con tu mensaje
this.titulo = Objects.requireNonNull(titulo, "El título es obligatorio");

// 2) record con constructor compacto: se ejecuta antes de asignar los campos
public record Persona(String nombre, int edad) {
    public Persona {
        if (edad < 0) throw new IllegalArgumentException("La edad no puede ser negativa");
    }
}

// 3) Lombok @NonNull: genera la comprobación de null en constructor y setter
@Getter @Setter @AllArgsConstructor
public class Categoria {
    private Long id;
    @NonNull private String nombre;
}
```

> **Cuidado con Lombok y con Jackson**
>
> `@AllArgsConstructor` y `@Setter` generan código **sin** tus validaciones (salvo `@NonNull`). Si necesitas validar, escribe tú ese constructor o ese setter. Y cuando Jackson convierte un JSON en un objeto usa el constructor vacío y los setters: lo que solo esté en un constructor con parámetros no se ejecuta.

3 En el DTO con Bean Validation

Cuándo: para comprobar la **forma** de lo que manda el cliente antes de que llegue al servicio: que venga, que no esté vacío, que tenga el tamaño y el formato correctos. No necesita consultar nada más.

Dependencia

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

DTO anotado y `@Valid` en el controlador

```
public class LibroPeticionDto {
    @NotBlank(message = "El ISBN es obligatorio")
    @Pattern(regexp = "[0-9]{13}", message = "El ISBN debe tener exactamente 13 cifras")
    private String isbn;

    @NotBlank(message = "El título es obligatorio")
    private String titulo;

    @NotBlank(message = "El autor es obligatorio")
    private String autor;

    @Min(value = 1, message = "Debe haber al menos un ejemplar")
    private int ejemplares;

    @NotNull(message = "La categoría es obligatoria")
    private Long categoriaId;
}

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Libro add(@Valid @RequestBody LibroPeticionDto dto) {   // sin @Valid no se valida nada
    return libroService.crear(dto);
}
```

Si algún campo no cumple, Spring **no llega a ejecutar el método** y lanza `MethodArgumentNotValidException`, que se traduce en un 400.

Anotaciones más usadas (`jakarta.validation.constraints`)

Anotación	Se usa en	Qué comprueba
`@NotNull`	Cualquier tipo	Que no sea `null`. Un `""` lo acepta.
`@NotEmpty`	Texto, listas	Ni `null` ni vacío. Un `" "` lo acepta.
`@NotBlank`	Texto	Ni `null`, ni vacío, ni solo espacios. La habitual para textos obligatorios.
`@Size(min, max)`	Texto, listas	Longitud del texto o número de elementos. Ej.: `@Size(max = 50)`.
`@Min` / `@Max`	Números enteros	Valor mínimo / máximo. Ej.: `@Min(1)`.
`@Positive` / `@PositiveOrZero`	Números	Mayor que 0 / mayor o igual que 0.

`@Negative` / `@NegativeOrZero`	Números	Menor que 0 / menor o igual que 0.
`@DecimalMin` / `@DecimalMax`	Decimales	Mínimo / máximo para precios e importes. Ej.: `@DecimalMin("0.01")`.
`@Digits(integer, fraction)`	Decimales	Número máximo de cifras enteras y decimales.
`@Pattern(regexp)`	Texto	Que cumpla una expresión regular (ISBN, DNI, código postal…).
`@Email`	Texto	Formato de correo electrónico.
`@Past` / `@PastOrPresent`	Fechas	Fecha anterior a hoy (fecha de nacimiento).
`@Future` / `@FutureOrPresent`	Fechas	Fecha posterior a hoy (fecha de devolución).
`@AssertTrue` / `@AssertFalse`	Booleanos	Que valga `true` / `false` (aceptar condiciones).
`@Valid`	Objeto o lista dentro del DTO	Hace que se validen también las anotaciones del objeto anidado.

Dos trampas habituales

1) Casi todas las anotaciones dan por **válido** un `null`: `@Size(max = 50)` o `@Min(1)` sobre un valor `null` no fallan. Si además es obligatorio, se combina con `@NotNull` o `@NotBlank`.
2) Con tipos primitivos (`int`) un campo que no llega vale `0`, no `null`: `@NotNull` no sirve. Usa `Integer` si necesitas distinguir «no enviado» de «cero».

Objetos anidados

```java
public class PrestamoPeticionDto {
    @NotNull @Valid                         // @Valid: valida también lo de dentro
    private SocioDto socio;

    @NotEmpty
    private List<@NotNull @Positive Long> librosIds;   // valida cada elemento
}
```

4 En el controlador (ruta y parámetros)

Cuándo: para lo que llega en la URL y no en el cuerpo. El controlador solo declara qué espera; no escribe `if`.

```java
@GetMapping("/{id}")
public Libro getLibroById(@PathVariable @Positive Long id) { ... }

@GetMapping
public List<Libro> listarLibros(
        @RequestParam(required = false) @Size(max = 100) String autor,
        @RequestParam(defaultValue = "0") @Min(0) int pagina) { ... }
```

Situación	Excepción de Spring	Código

`/api/libros/abc` (no es un número)	`MethodArgumentTypeMismatchException`	400
Falta un `@RequestParam` obligatorio	`MissingServletRequestParameterException`	400
JSON mal formado o con un tipo incorrecto	`HttpMessageNotReadableException`	400
El parámetro no cumple `@Positive`, `@Min`...	`HandlerMethodValidationException`	400
El cuerpo con `@Valid` no cumple	`MethodArgumentNotValidException`	400

En Spring Boot 3.2 o superior las anotaciones sobre parámetros funcionan directamente. En versiones anteriores hay que poner `@Validated` en la clase del controlador y la excepción es `ConstraintViolationException`.

5 En el servicio (reglas de negocio)

Cuándo: cuando para saber si algo es válido hay que mirar **otros datos** o el estado actual. Ninguna anotación del DTO puede saber si un ISBN ya existe: eso solo lo sabe el servicio, que tiene la lista (o el repositorio).

Tipo de regla	Ejemplo	Excepción propia	Código
El recurso existe	Buscar el libro 99 y no existe	`NotFoundException`	404
No se repite (unicidad)	ISBN o nombre de categoría que ya tiene otro	`ConflictoException`	409
La relación es válida	El `categoriaId` del libro no existe	`BadRequestException`	400
El estado lo permite	Prestar un libro sin ejemplares disponibles; superar el máximo de préstamos	`ConflictoException` o `BadRequestException`	409 / 400
Coherencia entre datos	Dejar menos ejemplares que libros prestados	`BadRequestException`	400

```
@Override
public Libro crear(LibroPeticionDto libro) {
    validar(libro);                          // forma del dato y categoría existente -> 400
    validarIsbnUnico(libro.getIsbn(), null);  // no repetido -> 409
    ...
}

private void validarIsbnUnico(String isbn, Long idIgnorado) {
    boolean isbnExiste = libros.stream()
            .anyMatch(l -> !l.getId().equals(idIgnorado) && l.getIsbn().equals(isbn));
    if (isbnExiste) {
        throw new ConflictoException("Ya existe un libro con el ISBN: " + isbn);
    }
}
```

- **Al modificar hay que ignorar el propio registro** (`idIgnorado`): si no, guardar un libro sin cambiarle el ISBN daría un 409 contra sí mismo.
- **Sin distinguir mayúsculas** cuando la regla lo pide: `equalsIgnoreCase`, y `trim()` para que «Novela » no cuente como distinto de «Novela».

- **El servicio no sabe nada de HTTP:** lanza excepciones propias, nunca devuelve un `ResponseEntity` ni un código de estado. La traducción a 404, 400 o 409 la hace el manejador global.
- **Los límites configurables** (máximo de préstamos, días) se leen de las propiedades; los que definen el modelo (ISBN de 13 cifras, nombre de 50) van como constantes con nombre, no como números sueltos.

> **¿En el DTO o en el servicio?**
> Las dos formas son válidas para la *forma* del dato (obligatorio, longitud, formato): con anotaciones en el DTO o con `if` en el servicio, como pide la Fase 1 de la biblioteca. Lo que **siempre** va en el servicio es lo que depende de otros datos. En proyectos reales se usan las dos a la vez: anotaciones para la forma y servicio para el negocio.

6 Validador propio

Cuándo: cuando ninguna anotación estándar sirve y la regla se repite en varios sitios. Se crea una anotación y una clase que la comprueba.

```
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = Isbn13Validator.class)
public @interface Isbn13 {
    String message() default "El ISBN debe tener exactamente 13 cifras";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

public class Isbn13Validator implements ConstraintValidator<Isbn13, String> {
    @Override
    public boolean isValid(String valor, ConstraintValidatorContext contexto) {
        return valor == null || valor.matches("[0-9]{13}");   // null lo decide @NotBlank
    }
}

// Uso en el DTO
@NotBlank @Isbn13
private String isbn;
```

Validar dos campos a la vez

Cuando la regla relaciona varios campos del mismo objeto (la fecha de fin posterior a la de inicio), la forma más simple es un método con `@AssertTrue` dentro del DTO:

```
@AssertTrue(message = "La fecha de devolución debe ser posterior a la de préstamo")
public boolean isFechasCoherentes() {
    return fechaPrestamo == null || fechaDevolucion == null
            || fechaDevolucion.isAfter(fechaPrestamo);
}
```

7 Grupos: crear y modificar

Cuándo: cuando el mismo DTO tiene reglas distintas según la operación (por ejemplo, la contraseña es obligatoria al crear pero no al modificar). Se usa `@Validated` en lugar de `@Valid` para indicar el grupo.

```java
public interface AlCrear {}
public interface AlModificar {}

public class SocioPeticionDto {
    @NotBlank(groups = {AlCrear.class, AlModificar.class})
    private String nombre;

    @NotBlank(groups = AlCrear.class)          // solo obligatoria al crear
    private String password;
}

@PostMapping
public Socio add(@Validated(AlCrear.class) @RequestBody SocioPeticionDto dto) { ... }

@PutMapping("/{id}")
public Socio modificar(@PathVariable Long id,
                       @Validated(AlModificar.class) @RequestBody SocioPeticionDto dto) { ... }
```

Si las diferencias son muchas, suele ser más claro tener dos DTO distintos.

8 En la configuración

Cuándo: para que la aplicación **no arranque** si alguien deja mal una propiedad, en lugar de fallar más tarde con un error raro.

```java
@Validated
@ConfigurationProperties(prefix = "biblioteca")
public record BibliotecaProperties(
        @NotBlank @DefaultValue("Biblioteca Decroly") String nombre,
        @Min(1) @DefaultValue("3") int maxPrestamos,
        @Min(1) @DefaultValue("15") int diasPrestamo) {
}
```

Con `biblioteca.max-prestamos=0` la aplicación se detiene al arrancar indicando qué propiedad está mal. `@DefaultValue` da el valor cuando la propiedad no está en el fichero.

9 En la base de datos

Cuándo: siempre que haya base de datos, como última barrera. Protege aunque alguien se salte el servicio o dos peticiones lleguen a la vez con el mismo ISBN.

```java
@Entity
public class Libro {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 13)
    private String isbn;

    @Column(nullable = false)
    private String titulo;

    @ManyToOne(optional = false)                // la categoría es obligatoria
    private Categoria categoria;
}
```

- `@Column(...)` crea restricciones en la tabla. Si se incumplen, salta `DataIntegrityViolationException`, que conviene traducir a 409.
- No sustituye a la validación del servicio: el mensaje de la base de datos no es claro para el cliente y llega tarde. Se usan las dos.
- Las anotaciones de Bean Validation puestas en una entidad también se comprueban automáticamente antes de guardar.

10 Qué pasa cuando falla: errores HTTP

Todas las validaciones acaban en una excepción. Un único manejador global las traduce a una respuesta `ProblemDetail` con su código.

```
@RestControllerAdvice
public class GlobalErrorHandler {

    @ExceptionHandler(NotFoundException.class)
    public ProblemDetail notFound(NotFoundException e) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, e.getMessage());
        problem.setTitle("Recurso no encontrado");
        return problem;
    }

    // Errores de @Valid: se devuelve qué campo falla y por qué
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail datosNoValidos(MethodArgumentNotValidException e) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                HttpStatus.BAD_REQUEST, "Hay campos que no son válidos");
        problem.setTitle("Petición errónea");
        Map<String, String> errores = new LinkedHashMap<>();
        e.getBindingResult().getFieldErrors()
                .forEach(error -> errores.put(error.getField(), error.getDefaultMessage()));
        problem.setProperty("errores", errores);
        return problem;
    }
}
```

Qué ha pasado	Excepción	Código
Datos no válidos (forma)	`BadRequestException`, `MethodArgumentNotValidException`	**400** Bad Request
El recurso no existe	`NotFoundException`	**404** Not Found
Dato repetido o estado que lo impide	`ConflictoException`	**409** Conflict
Fallo del propio programa	Cualquier otra	**500** (no debería ocurrir)

- Las excepciones propias no necesitan `@ResponseStatus`: el código lo decide solo el manejador.
- Con `spring.mvc.problemdetails.enabled=true`, los errores que genera Spring (JSON mal formado, `/libros/abc`) salen también como `ProblemDetail` sin escribir más handlers.
- «No encontrado» es 404. El 204 es una respuesta **correcta** sin contenido (el borrado), no un error.

11 Chuleta final y errores típicos

Quiero validar…	Dónde	Con qué
Que un objeto no pueda existir sin edad válida	Constructor (y setter)	`if` + excepción
Que el nombre venga y no esté vacío	DTO (o servicio)	`@NotBlank`
Que el nombre tenga 50 caracteres como máximo	DTO (o servicio)	`@Size(max = 50)`
Que el ISBN tenga 13 cifras	DTO (o servicio)	`@Pattern` o validador propio
Que haya al menos 1 ejemplar	DTO (o servicio)	`@Min(1)`
Que el correo tenga formato de correo	DTO	`@Email`
Que la fecha de nacimiento sea pasada	DTO	`@Past`
Que el id de la ruta sea un número positivo	Controlador	`@PathVariable @Positive`
Que el ISBN o el nombre no se repitan	**Servicio**	`ConflictoException` → 409
Que la categoría del libro exista	**Servicio**	`BadRequestException` → 400
Que el libro que se pide exista	**Servicio**	`NotFoundException` → 404
Que no se presten más libros del máximo	**Servicio** + propiedades	Excepción propia
Que la fecha de fin sea posterior a la de inicio	DTO	`@AssertTrue` en un método
Que el máximo de préstamos configurado sea ≥ 1	Configuración	`@Validated` + `@Min(1)`
Que nunca haya dos ISBN iguales guardados	Base de datos	`@Column(unique = true)`

Errores típicos

- Olvidar `@Valid` en el controlador: las anotaciones del DTO no hacen nada.
- Poner `@NotNull` en un texto: deja pasar `""` y `" "`. Para textos, `@NotBlank`.
- Poner solo `@Size` o `@Min` y dar por hecho que el campo es obligatorio: un `null` pasa.
- Comprobar reglas de negocio en el controlador en vez de en el servicio.
- Validar «no repetido» al modificar sin excluir el propio registro.
- Devolver 400 para un dato repetido (es 409) o 204 para «no encontrado» (es 404).
- Validar solo en el constructor un atributo que tiene setter.
- Escribir el límite como número suelto en mitad del código en vez de una constante o una propiedad.